

Architecture & Ecosystem Design —

Part 5: Domain Model & Bounded Contexts

SUMMARY

The AROH ecosystem should be organized around business domains rather than technologies, frameworks, or databases. Every domain represents a distinct business capability with clear responsibilities, ownership, and boundaries. A domain should encapsulate its own entities, rules, workflows, policies, events, and services without leaking internal implementation details to other parts of the ecosystem. The objective is to ensure that every major business capability evolves independently while interacting with other capabilities only through well-defined contracts.

DOMAIN-DRIVEN PHILOSOPHY

A bounded context owns its data. A bounded context owns its rules. A bounded context owns its terminology. A bounded context owns its events. Other domains interact through interfaces rather than direct implementation.

CORE DOMAIN MAP

Identity; Authentication; Authorization; Profile; Membership; Wallet; Registry; Products; Configuration; Notifications; Analytics; AI; CMS; Search; Audit; Monitoring; Feature Flags; Localization; Media; Billing (Future); Marketplace (Future); Organizations (Future); Plugins (Future); Workflow Automation (Future). Every future capability should belong to one existing domain or justify the creation of a new bounded context.

IDENTITY DOMAIN

Identity represents the user. Owns: User ID; Authentication Identity; Email; Verification State; Linked Accounts; Session Metadata; Security Metadata; Authentication History; Identity Events; Identity Policies; Identity Lifecycle. Never owns: Wallet; Membership; Notifications; Analytics; Product Data. Identity is the root domain from which every other user-related capability references the user.

AUTHENTICATION DOMAIN

Authentication answers: Who is this user? Owns: Credentials; Authentication Tokens; Session Validation; Login; Logout; Registration; Password Reset; Provider Integration; Remember Me; Session Expiration; Authentication Logs; Authentication Events. Authentication never decides permissions; that belongs to Authorization.

AUTHORIZATION DOMAIN

Authorization answers: What can this user do? Owns: Roles; Permissions; Claims; Policies; Access Rules; Feature Access; Admin Rights; Operator Rights; Ownership Rules; Permission Evaluation; Future RBAC; Future ABAC; Future Organization Policies. Authorization should never know business workflows, only permissions.

PROFILE DOMAIN

Profile represents the user's public and personal information. Owns: Display Name; Avatar; Biography; Timezone; Language; Preferences; Theme; Accessibility; Notification Preferences; Personal Settings; Privacy Settings; Recent Activity; Bookmarks; Achievements; Connected Products. Profile does not own authentication.

MEMBERSHIP DOMAIN

Membership owns ecosystem access. Owns: Plans; Benefits; Entitlements; Subscription State; Upgrade Rules; Downgrade Rules; Expiration; Renewals; Grace Period; Promotions; Discount Rules; Future Trials; Future Enterprise Plans. Products ask: Does this user have access? Membership answers.

WALLET DOMAIN

Wallet represents financial integrity. Owns: Balance; Ledger; Transactions; Credits; Debits; Refunds; Rewards; Purchases; Membership Charges; Future Marketplace Transactions; Future Billing; Future Transfers; Future Coupons; Future Promotions. Wallet is authoritative. No product modifies balances directly.

REGISTRY DOMAIN

Registry knows every product. Owns: Metadata; Categories; Visibility; Availability; Dependencies; Documentation Links; Navigation; Icons; Descriptions; Version Compatibility; Supported Platforms; Search Metadata. Registry enables ecosystem discovery.

PRODUCT DOMAIN

Every product owns: Business Logic; Features; Domain Models; Specialized Services; Product Assets; Product Navigation; Reports; Settings; Documentation. Products never own platform infrastructure.

CMS DOMAIN

CMS manages ecosystem content. Owns: Announcements; News; Updates; Maintenance Notices; Release Notes; Future Blogs; Future Documentation Publishing; Future Landing Content; Scheduling; Publishing Rules; Editorial Workflow; Approval Flow; Content Lifecycle. CMS serves every product.

ANALYTICS DOMAIN

Analytics transforms events into knowledge. Owns: Usage Events; Business Metrics; Feature Metrics; Retention; Funnels; Growth; Platform Metrics; Product Metrics; AI Metrics; Wallet Metrics; Membership Metrics; Performance Reports; Future Recommendations. Analytics stores observations, not business logic.

AI DOMAIN

AI acts as a platform capability. Owns: Prompt Routing; Provider Selection; Conversation Context; Knowledge Retrieval; Prompt Templates; Tool Integration; Model Selection; Caching; Rate Limits; Cost Tracking; Memory; Future Agents; Future Automation; Future Personalization. AI should never belong to a single product.

NOTIFICATION DOMAIN

Owns: Email; Push; In-App; Announcements; Scheduling; Templates; Read Status; Preferences; Retry Logic; Delivery Status; Priority; Localization; Future SMS; Future Webhooks; Future Workflow Alerts.

CONFIGURATION DOMAIN

Owns: Runtime Settings; Feature Flags; Environment Config; Limits; Policies; Platform Defaults; Pricing; Maintenance Mode; Regional Settings; AI Configuration; Analytics Configuration. Products inherit configuration.

SEARCH DOMAIN

Owens: Indexing; Search Results; Ranking; Suggestions; Autocomplete; Commands; Documentation Search; Product Search; Global Search; Future Semantic Search; Future AI Search. Search consumes metadata, never business logic.

AUDIT DOMAIN

Owens: Audit Logs; Security Events; Administrative Actions; Permission Changes; Wallet Events; Configuration Changes; Content Changes; System History; Compliance Records; Future Legal Reports. Everything important should be auditable.

MONITORING DOMAIN

Owens: Health Checks; Performance; Availability; Errors; Tracing; Infrastructure Metrics; API Metrics; Database Metrics; Future Incident Detection; Future Auto Recovery. Monitoring observes, never modifies.

FEATURE FLAG DOMAIN

Owens: Experiments; Rollouts; Beta Features; Canary Releases; Regional Features; Membership Features; Developer Features; Emergency Disable; Future A/B Testing; Future Progressive Rollout.

LOCALIZATION DOMAIN

Owens: Languages; Translations; Regional Formats; Currency; Timezones; Date Formats; Number Formats; RTL Support; Accessibility Localization; Future Regional Content. Localization should remain centralized.

MEDIA DOMAIN

Owens: Images; Videos; Documents; Icons; Logos; Uploads; Optimization; Compression; Storage; CDN; Future Streaming; Future Versioning. Products reference media, not store it independently.

BILLING DOMAIN (FUTURE)

Owens: Payments; Invoices; Taxes; Refund Policies; Payment Providers; Subscriptions; Financial Reports; Receipts; Billing Events; Billing APIs. Wallet and Billing remain separate. Wallet manages ecosystem value. Billing manages external money.

MARKETPLACE DOMAIN (FUTURE)

Owens: Listings; Purchases; Reviews; Partners; Extensions; Transactions; Downloads; Licensing; Marketplace Analytics; Future Revenue Sharing. Marketplace should consume Wallet and Billing, not replace them.

ORGANIZATION DOMAIN (FUTURE)

Owens: Teams; Organizations; Projects; Permissions; Shared Resources; Members; Roles; Invitations; Enterprise Features; Future SSO; Future SCIM.

PLUGIN DOMAIN (FUTURE)

Owens: Plugin Registry; Installation; Updates; Compatibility; Permissions; Sandbox; Developer APIs; Lifecycle; Extension Events; Future Marketplace Integration.

DOMAIN COMMUNICATION

Domains communicate through events and contracts. Identity Created -> Profile Created, Membership Assigned, Wallet Created, Notifications Sent, Analytics Recorded. No domain directly manipulates another domain's internal data. Each domain publishes events; interested domains subscribe. This reduces coupling, improves scalability, and supports future event-driven architecture.

DOMAIN OWNERSHIP RULES

Every entity has exactly one owner. Every rule has exactly one owner. Every workflow has exactly one owner. Every event has exactly one publisher. Duplicate ownership is prohibited. If two domains require identical logic, that logic should be extracted into a shared platform capability rather than copied.

LONG-TERM DOMAIN EVOLUTION

As AROH expands, new capabilities should emerge by extending existing bounded contexts rather than introducing arbitrary modules. The domain model should remain stable even as infrastructure changes. Databases, frameworks, cloud providers, and implementation technologies may evolve, but the business domains—Identity, Wallet, Membership, AI, Analytics, Notifications, Registry, Configuration, Search, CMS, Audit, and future enterprise capabilities—should continue serving as the permanent conceptual foundation of the ecosystem. A stable domain model ensures that architecture remains understandable, maintainable, and scalable throughout the lifetime of the platform.